LLM AGENT 注入攻击检测 · 阶段性研究报告

v3 化简表示：事件节点 / 两类边 / 三问输出

4 类事件节点（U / Th / AC / TR）+ 两类可并存的边（时间先后 E1 / 信息传导 E2）；P1–P8 不再是预先枚举的模板，而是处理流程跑完后回答「三问」时给出的压缩标签 —— 输出契约、检测口径与跨数据集验证

报告版本 v3.4（表示 = 节点 + 两类边；判定输出 = 三问契约，P1–P8 降级为压缩标签）

数据集 AgentDrift test 1,881 条 | MemEvoBench Workflow 320 条记忆

检测口径 结构判据 a2 + P4 + s3p（+ 低召回子型补丁 P6/P7/P8）

日期 2026-10-15

结构 F1 0.908（P1–P8） 合并 F1 0.902（LLM ∨ P8） 零新增误报 重构校验 2,201 条 0 差异

目录

1. 摘要
2. 背景与问题
3. 第一节 · 事件节点定义（U / Th / AC / TR）
4. 第一节 · 关联边定义（E1 时间先后 / E2 信息传导，可并存）
5. 第二节 · 三问输出契约（表示 → 恶性结果 / 源点 / 演化）
   5.1 节点属性　5.2 两类边与两个节点侧谓词　5.3 三问输出契约　5.4 压缩标签 P1–P8（Q3 形状别名）　5.5 实测节点模式分解　5.6 攻击家族 → 多标签映射
6. 数据集与评估口径
7. 结果：AgentDrift 全量重测
8. 归因：漏报来自哪、P8 为何命中
9. 探针验证：LLM 判读端对齐
10. 结构化合并：LLM ∨ 结构 P8
11. 决策记录与再现性
12. 经验与教训
13. 附录：表示、三问输出与校验的代码落地

1 摘要

本报告给出注入攻击检测表示体系的 v3 化简定义及其在 AgentDrift 与 MemEvoBench 上的完整验证。核心是把复杂的攻击语义**收敛为 4 类事件节点 + 2 类可并存的关联边**（时间先后 / 信息传导）；判定流程对每条轨迹回答**三个问题**——是否产生恶性结果、攻击源点在哪、从源点如何演化到结果。沿用中的 P1–P8 **不再是预先枚举的模板，而是三问输出（尤其是 Q3 演化路径）的压缩标签。**

4	2	3	0.908
事件节点类型（U/Th/AC/TR）	可并存的边（E1 时间先后 / E2 信息传导）	输出契约（Q1 恶性结果 / Q2 源点 / Q3 演化）	AgentDrift 结构 F1（指标不变）

主要结论

1. **表示收敛成立。**权限变化、资源流转、上下文诱导、推理传导等区分，全部可下沉为节点属性；模板判定所需信息由「逐 `steps` 遍历 + 标签」改为「节点属性 + 边」的表达，离线校验在 2,201 条轨迹上**判定/模板/图统计 0 差异。**
2. **三问输出契约与既有判定逐条等价。**新增薄派生层 `_v3_analysis.py`，仅由既有判定结果做步号算术：**Q1**（outcome = 已落地 / 未遂 / 无 + 危险动作性质）≡ 原 `hit`；**Q2**（primary 源点 + secondary 二次注入点）的源点 ≡ `min(signal_steps)`；**Q3**（源→结果的显式路径 + 形状）与 P1–P8 一一对应。2,201 条上三条等价**全部成立**，检测计算零改动、零 API。
3. **低召回子型补丁有效。**新增 P6（摄取传导）、P7（危险执行）、P8（伪装系统指令），仅在 P1–P5 未命中时启用。AgentDrift 结构 F1 **0.849 → 0.908**，精确率同向改善（0.930 → 0.937），**825 条负样本零新增误报**；MemEvoBench 结构 F1 **0.294 → 0.678**（ingestion 召回 7.3% → 92.7%）。
4. **结构端与 LLM 判读端可低成本对齐。**采用后置 OR（`final = LLM_pred ∨ structural_P8`），零 API 调用下合并 F1 **0.874 → 0.902**，召回 0.875 → 0.925，误报 0 新增。

> **一句话总结：**表示层只有 4 类事件节点和两类可并存的边；对每条轨迹回答「是否产生恶性结果 / 注入源点（含二次注入点）在哪 / 从源点如何演化到结果」三问。P1–P8 只是 Q3 演化路径的**压缩标签**，不再是需要预先枚举的模板——叙事更简，且既有性能一项不丢。

2 背景与问题

2.1 威胁模型

LLM Agent 在工具调用轨迹中执行任务：读取用户指令、检索外部内容（网页 / 邮件 / 表单 / 记忆）、推理规划、调用工具、接收工具返回。攻击者在**外部内容**中植入指令（prompt injection），诱导 Agent 偏离原任务，产生**未授权外发**（数据泄露到外部目标）或**有害执行**（危险载荷落地执行）。

2.2 需要回答的三个问题

- **表示**：轨迹中哪些要素是"本质"的？能否用极少的节点/边类型覆盖全部攻击与良性行为？
- **判定**：如何从表示中稳定识别攻击？既要高精确率（负样本零误报），又要尽可能高的召回。
- **对齐**：结构化判据与 LLM 判读端如何在不大量消耗 API 的前提下取得一致？

2.3 前序工作与遗留问题

早期模板用**抽象角色**描述轨迹：不写具体节点类型，而把节点抽象成少数几个固定角色，再写角色之间的链式关系。记号简洁，但存在三个问题：

- **与实现脱节。**代码里已构建节点 U/Th/AC/TR 与边 E1/E2，但模板判定仍直接遍历 `steps` 并按 step 索引与 `label` 计算，图只进入报告统计（"装饰性"）。
- **表达力单一。**旧记号假定"一种攻击 = 一条线性链"，难以表达"一种攻击类型有多个典型模板"以及"信息摄取型"（无独立外发步、注入藏在观察/记忆中）等形态。
- **角色抽象掩盖节点类型差异。**同一个抽象角色在实现里可能落在三种不同节点类型上（用户指令被污染 / 观察被污染 / 推理被内化），把三者抹成一种，掩盖了它们其实是不同的模板。

本报告即针对上述三点，完成**属性下沉 → 模板节点原生枚举 → 离线零差异校验**的改造。

3 第一节 · 事件节点定义（U / Th / AC / TR）

轨迹被建模为一个**有向图**，节点是"事件"，边是"事件之间的关联"。节点只保留 4 类基础类型，其余区分全部下沉为**节点属性**。

表 3-1 4 类基础事件节点

节点	含义	来源字段	承载属性（下沉）
U	用户指令	`task`（前置，step = −1）	`inducement`
Th	推理 / 思考	`step.thought`	`inducement`、`refuse`、`sys`、`label`
AC	工具调用（动作）	`step.tool` + `step.args`	`tool`、`args`、`action_type`、`resource_type`、`unauth_ext`、`harm`、`branch3`、`exec`、`mem`、`label`
TR	工具返回（观察）	`step.obs`	`inducement`、`danger`、`novel`、`label`

3.1 为什么是这 4 类

对 AgentDrift 12,536 条轨迹（225,608 个节点事件）与 MemEvoBench 83 簇做全量要素拟合，得到三条化简结论：

- **四类节点即可覆盖全量要素。** MemEvoBench 表示 sanity 中 4 类节点对轨迹要素覆盖率为 **True**（节点计数 `{U:320, AC:848, TR:320}`）。
- **其余区分是"属性"而非"类型"。** 权限变化 / 资源流转（R1–R6）/ 上下文诱导 / 推理传导，都可作为节点上的布尔或枚举属性承载。
- **属性是判定所需的最小信息。** 模板判定只读取节点属性，不再回读原始 `steps`。

属性取值口径

表 3-2 节点属性的计算口径

属性	节点	口径
`inducement`	U/Th/AC/TR	文本命中诱导语正则（`_struct_v2.INDUCE`）
`refuse`	Th	推理文本命中拒绝/警觉正则（`_struct_v2.REFUSE`）
`sys`	Th	推理文本把注入当系统级指令内化（`RE_SYS`，见 5.4）
`unauth_ext`	AC	动作外发（`outb`）且参数含 ∉ **W** 的外部主机
`harm`	AC	载荷拆类：武器类命中，或凭证类命中且（外发 ∨ 外部目标）
`branch3`	AC	动作类别落入门槛 `GATE`（a2 = A2..A6）且非显式豁免
`exec`	AC	工具名属执行类（terminal_execute / run_command …）
`mem`	AC	工具名属记忆检索类（recall_memory / retrieve_memory …）

属性	节点	口径
`danger`	TR	观察文本含危险载荷（shell 破坏 / 外联下载 / 凭证）
`novel`	TR	载体引入"任务未提及实体"（专名 / 编号 / 邮箱 / URL / 长 ID）

W = 公司域 ∪ 任务实体 ∪ 任务点名站点 ∪ 任务 URL（可信边界）。

4 第一节 · 关联边定义（E1 / E2）

边只保留 2 类，刻画节点之间最本质的两种关联；**同一对节点可并存两类边。**例如一条用户指令既**传递了信息**（其中的邮箱/URL 经 E2 流向后续动作），又**触发了 Agent 的思考与动作**（经 E1 指向后继 Th/AC）——两件事同时成立，就挂两条边，互不挤占。"时间先后"取**相邻骨架**而非全序：任意两点都有先后，全序边不携带额外信息，还会稀释"直接触发"。

表 4-1 2 类关联边

边	含义	判定
E1	**时间先后**（相邻节点骨架）	节点序列中相邻的两个节点直接相连：既含 `U(step=-1) → 首事件`，也含步内 `Th → AC → TR`；回答"谁先谁后、谁直接触发谁"
E2	**信息（数据）传导**（类型化实体交集）	节点文本的**类型化实体**交集非空：email / url / num / id（strict 档）；回答"什么数据从谁流向了谁"

4.1 E2 的抗噪设计

信息传导只承认**类型化实体**的交集，而不是任意词重叠。这样"同一封邮件地址 / 同一个 URL / 同一个编号 / 同一个 ID 从载体流向下游动作"才是传导，避免通用词造成的假阳性。

```
E2(i, j) ⇔ ents(text_i) ∩ ents(text_j) ≠ ∅
ents(·) = { (email, e) } ∪ { (url, u) } ∪ { (num, n) } ∪ { (id, t) }
其中 id 仅收录含数字 / 下划线 / 连字符的 token（天然抗噪）
```

4.2 建图规模（AgentDrift 每轨迹均值）

17.83	37.04	16.83	20.21
节点 / 轨迹	边 / 轨迹	E1 / 轨迹	E2 / 轨迹

2.82

并存节点对 / 轨迹

"并存节点对" = 同一节点对同时存在 E1 与 E2；说明时间先后与信息传导是两个正交维度（每轨迹均值 2.82 对）。

5 第二节 · 三问输出契约（从表示到判定输出）

> **本章要点**：表示层只有两种构件——**节点**（U / Th / AC / TR，其余区分全部下沉为节点属性）与**两类可并存的边**（E1 时间先后 / E2 信息传导）。除此之外不再预设任何"风险模板分类"。判定流程跑完后，对每条轨迹回答**三个问题**：
> **Q1** 当前执行路径是否产生了恶性结果；
> **Q2** 攻击的发起源点在哪里（以及次要 / 二次注入点）；
> **Q3** 从注入到结果事件是如何演化的。
> 沿用中的 **P1–P8 只是 Q3 演化路径的压缩标签**（为与历史指标连续而保留），不是需要预先枚举的模板分类。

5.1 判定用节点属性（由第 3 章下沉而来）

表 5-1 模板判定直接读取的节点属性

节点	属性
U	`inducement`
Th	`inducement`、`refuse`、`sys`、`label`
AC	`unauth_ext`、`harm`、`branch3`、`exec`、`mem`、`args`、`label`
TR	`inducement`、`danger`、`novel`、`label`

5.2 两个节点侧谓词（在节点上定位"源点"与"危险动作"）

三问不靠额外分类，只在节点类型与属性上求两个**节点侧谓词**，分别给 Q2 提供"注入源点"、给 Q1 提供"恶性结果落点"；边只负责把这些节点按时间与数据串成 Q3 的路径：

表 5-2 定位源点 / 危险动作的节点侧谓词与路径形状

要素	定义（直接落在节点类型与属性上）
谓词 A · 带信号节点	节点 `n`，类型 ∈ {`U`, `Th`, `TR`}，满足：`text(n)` 含外部主机 ∉ W，或命中诱导语（`inducement`）；**且信号在下游复现**——该 host / 诱导语 / 新颖 token 出现在某个 `step > n` 的 `AC.args` 或 `Th.text`
谓词 B · 危险动作节点	节点 `m`，类型 == `AC`，且 `unauth_ext` ∨ `harm` ∨ `branch3`
相邻 `—E1→`	`step(m) − step(n) == 1`（E1 步骨架的最小粒度）
隔步	`step(m) − step(n) ≥ 2`（中间存在其它节点）
良性节点	节点 `label == benign`
`Th[refuse]`	Th 节点上 `refuse` == True（拒绝 / 警觉）

> **注意**："信号在下游复现"是独立谓词，**不等于**"存在 E2 边"。E2 只认类型化实体交集（email / url / num / id）；诱导语复现属词法复现，不产生 E2 边。二者同族但不等价，混用会重蹈"图只作装饰"的问题。

5.3 三问输出契约

流程对每条轨迹输出一个三问结果对象（`_v3_analysis.py` 的 `analyze(rec)`）。三个答案**全部由既有判定结果做步号算术派生**，不引入任何新判定、不调用 API：

表 5-3　三问输出契约（字段与取值）

问	字段	含义 / 取值
Q1 恶性结果	`outcome`	`realized` 已落地危险动作 / `attempted` 注入已起意但被拒绝（未遂）/ `none` 路径无恶性结果
	`sink_step / sink_reason`	首个落地危险动作的步号与性质：`c_ext` 未授权外发 · `c_harm` 敏感/有害载荷 · `branch3` 敏感动作未授权
	`severity`	危险动作的人类可读性质；P6/P7/P8 路径无谓词 B 落点时取机制性质（摄取落地 / 危险执行 / 系统指令被采纳）
Q2 源点	`primary`	最早携带注入信号的节点 `{step, nid, type, basis}`；`basis ∈ {signal, novel_carrier, danger_payload, sys_internalized}`，分别对应谓词 A 信号、记忆/载体新实体、危险载荷、伪装系统指令
	`secondary[]`	其余注入承载节点 = **次要 / 二次注入点**（如同一轨迹中第二个被污染的观察 TR、多个误导记忆载体）
Q3 演化	`shape`	源→结果的路径形状：`adjacent` 相邻即落地 / `gapped` 中间夹良性步 / `blocked` 被拒绝 / `flow` 经信息传导摄取 / `exec` 危险载荷被执行 / `sys` 系统指令内化 / `no_chain` 无攻击链
	`source_step, sink_step, gap`	路径两端步号与中间间隔步数（`gap = sink − source − 1`）
	`path_nids, flow_links, label`	源→汇之间的显式节点序列（E1 串起）、区间内的 E2 信息传导边；`label` 即该路径的压缩标签 P1–P8

960 / 31 / 890
AgentDrift Q1：已落地 / 未遂 / 无

926 / 94
Q2 primary 载体：signal / sys 内化

568 / 288 / 31 / 104
Q3：相邻 / 隔步 / 被拒 / 系统指令

232 / 11
MemEvoBench Q3：信息摄取 / 危险执行

另有无攻击链 `no_chain`：AgentDrift 890（P5 861 + signal_only 29）、MemEvoBench 32（P5）。Q3 形状与 P1–P8 在 2,201 条上一一对应（等价校验 3，见 13.2）。

5.4 压缩标签 P1–P8（Q3 演化形状的别名）

P1–P8 不增加任何表示构件，只是给 Q3 的 `shape` 与 Q1 的 `outcome` 组合起的**短名**，用于与历史指标和 LLM 判读端连续。映射关系：

表 5-4　压缩标签 → 三问取值映射

标签	Q1 outcome	Q2 primary 的 basis	Q3 shape	含义

标签	Q1 outcome	Q2 primary 的 basis	Q3 shape	含义
P1	realized	signal	adjacent	带信号节点与危险动作相邻（step 差 1）
P2	realized	signal	gapped	两者之间夹其它节点（step 差 ≥ 2）
P3	realized	signal	adjacent	P1 且危险动作之后回归良性步
P4	attempted	signal	blocked	有信号、无下游危险动作，但出现 Th[refuse]
P5	none	—	no_chain	无带信号节点
P6	realized	novel_carrier	flow	载体 TR[novel] 的实体经信息传导流入下游 AC.args
P7	realized	danger_payload	exec	TR[danger] 的危险载荷被下游 AC[exec] 执行
P8	realized	sys_internalized*	sys	Th[sys]：注入被当系统级指令内化
signal_only	none	signal	no_chain	有信号但下游既无危险动作也无拒绝（悬空）

* 104 条 P8 中有 10 条在更早的步上已存在谓词 A 信号节点，此时 Q2.primary 如实记为更早的 `signal`，`sys` 节点进入 secondary——源点归属服从"最早"，不因标签是 P8 而改写。

标签的**具体赋值规则**与历史口径完全一致（只是换了个读法）：

5.4.1 路径标签 P1–P5

```
P1  n ∈ {U,Th,TR}[谓词 A] —E1→ m ∈ AC[谓词 B]
    检出：带信号节点 n 与危险动作节点 m 的 step 差 == 1（相邻）。
    三种节点侧形态分别枚举：U[signal] —E1→ AC[…]、Th[signal] —E1→ AC[…]、TR[signal] —E1→ AC[…]

P2  n ∈ {U,Th,TR}[谓词 A] —(step 差 ≥ 2)→ m ∈ AC[谓词 B]
    检出：n 与 m 之间存在其它节点（隔步）。

P3  P1 的任一模式 ∧ ∃ 节点 k，step(k) > step(m) 且 label(k) == benign
    检出：相邻命中，且危险动作之后仍有良性节点（攻击后回归正常流程）。

P4  ∃ n[谓词 A] ∧ ∄ m[谓词 B] ∈ 下游(n) ∧ ∃ Th[refuse] ∈ 下游(n)
    检出：有带信号节点、下游无危险动作节点，但出现拒绝/警觉 → 未遂（failed_attempt）。

P5  ∄ n[谓词 A]
    不命中：不存在带信号节点（no_signal）。
```

其中 `下游(x)` 取 `step > step(x)` 的全部节点。读作三问即：P1–P3 的 Q1=realized、P4 的 Q1=attempted、P5 的 Q1=none。

5.4.2 补丁标签 P6–P8

P1–P5 覆盖"带信号节点 → 危险动作节点"的路径型攻击，对**信息摄取型**（注入藏在网页 / 表单 / 记忆，无外发参数）失明。补三条按节点类型直接写出的机制型标签；它们**仅在 P1–P5 路径未形成时启用**，不改变 P1–P5 语义。

P6 摄取传导型 ∃ AC[mem] ∧ ∃ TR[novel]，
且 novel 实体出现在 U_{step > 该 TR} AC.args
记忆节点存在（记忆介导）；载体 TR 引入"任务未提及实体"；
该实体被下游动作参数使用（实体在 AC.args 复现）。
效果：对非记忆轨迹零触发。

P7 危险执行型 ∃ TR[danger]，∃ AC[exec]（step > 该 TR）
某工具返回节点含危险载荷（shell 破坏 / 外联下载 / 凭证），
且下游存在执行类动作节点 → 载荷被执行。

P8 伪装系统指令型 ∃ Th[sys]
某步推理节点把注入内容当作系统级指令内化，是部分注入（注入藏于观察、无独立带信号节点）被 Agent 采纳的语言痕迹。
判据（RE_SYS）：

```
\b(the\s+)?system\s+(suggest|recommend|advis|instruct|note|
                     require|request|say|state|indicat|prompt)
```

> **为何 P8 是"机制签名"而非统计巧合**：attacked_partial 的注入溶解在观察文本中（无独立带信号节点）→ 结构端判 P5、LLM 判 P5 漏检；但注入措辞伪装为"系统指令"，Agent 在 thought 中把它内化为"系统建议/要求"，从而留下 `sys` 语言痕迹。该痕迹**只在 partial 注入场景产生**，其余攻击子型与全部 825 条负样本**零触发**。

5.5 实测节点模式分解（数据导出）

把 P1–P3 的命中按 **带信号节点类型 × 其属性 × 危险动作属性 × 路径形状**逐条分解（`_v3_pattern_decomp.py`，零 API），得到的是数据导出的模式清单，而非记号推演：

表 5-5　P1–P3 命中的节点模式分解（AgentDrift 1,881）

信号节点	信号属性	动作属性	路径形状	命中
TR	外部域 + 诱导语	`branch3`	相邻	255
TR	外部域 + 诱导语	`c_ext`	相邻	163
TR	外部域 + 诱导语	`branch3`	隔步	145
TR	诱导语	`branch3`	相邻	132
TR	诱导语	`branch3`	隔步	71
TR	外部域 + 诱导语	`c_ext`	隔步	23
TR	外部域	`branch3`	隔步	22
TR	外部域	`branch3`	相邻	12
TR	外部域	`c_ext`	隔步	7
TR	任务级	`branch3`	隔步	6

信号节点	信号属性	动作属性	路径形状	命中
TR	外部域 + 诱导语	`c_harm`	隔步	5
TR	诱导语	`c_ext`	隔步	4
TR	诱导语	`c_harm`	隔步	3
TR	外部域	`c_ext`	相邻	3
TR	外部域 + 诱导语	`c_harm`	相邻	2
TR	任务级	`c_ext`	隔步	2
TR	诱导语	`c_ext`	相邻	1

856
P1–P3 命中；带信号节点 100% 为 TR

643 / 203 / 10
动作属性 branch3 / c_ext / c_harm

568 / 288
路径形状 相邻 / 隔步

0
带信号节点落在 U 或 Th 的次数

数据结论：本数据集上 P1–P3 的**带信号节点 100% 落在** `TR`（工具返回 / 观察），`U` 与 `Th` 从未承担该谓词——攻击的实质是"**观察被污染**"，这正是 attacked_partial 的结构本质。另：P5（无带信号节点，861 条）中有 **247 条**的 `Th` 携带诱导语、**91 条**的 `TR` 携带诱导语，只是信号未向下游传导，构成当前的结构性盲区。

5.6 攻击家族 → 多标签映射

因为标签按节点类型与演化形状给出，同一种攻击类型自然对应多个具体标签（即 Q3 的不同演化形状）：

表 5-6　攻击家族的多标签映射

攻击家族	典型标签	机制（Q2 源 → Q3 演化）
external_send（外部外发）	P1 P2 P3 P8	`TR[谓词 A]` —E1/隔步→ `AC[谓词 B]`；部分变体注入被内化为系统指令（`Th[sys]`）
ingestion（信息摄取）	P6	`AC[mem]` + `TR[novel]` → 实体流入下游 `AC.args`
exec（危险执行）	P7	`TR[danger]` → 下游 `AC[exec]`
failed（未遂）	P4	`TR[谓词 A]` 存在、下游无 `AC[谓词 B]`，但出现 `Th[refuse]`

标签赋值的实现约定：先尝试 P1–P5 的路径形状；路径未形成时按 `P7 → P6 → P8` 顺序赋机制标签（P7 机制最明确、P6 有记忆介导门控、P8 为语言痕迹，故 P8 置于最后，避免抢占更确定的机制判定）。这只是 Q3 `shape` 的赋值优先级，不新增任何节点或边类型。

6 数据集与评估口径

表 6-1 数据集

数据集	规模	正类	负类	说明
AgentDrift test	1,881	attacked 831 + failed_attack 225 = **1,056**	benign 600 + hard_negative 225 = **825**	子型：attacked_full / partial / delayed
MemEvoBench Workflow	320 条记忆（83 簇）	misleading	correct	记忆级：外发子型 / 摄取子型
MemEvoBench QA	324 条误导记忆	misleading	correct	无工具动作 → 结构不可达（见 7.4）

6.1 检测口径（定稿配置）

- `gate = a2`：`c_ext` 不受动作门槛限制；`branch3` 门槛为 A2..A6。
- `W` = 公司域 ∪ 任务实体 ∪ 任务点名站点 ∪ 任务 URL。
- `c_exp = off`、Th 不扫、载荷拆类（split_harm）开。
- P4 未遂开、`s3p` 参数级载荷传导开。
- 低召回补丁 `use_p6 = use_p7 = use_p8 = True`。

6.2 指标口径

正类 = attacked + failed_attack；负类 = benign + hard_negative。检出 = 命中 P1–P8 中任一模版。**精确率优先**：任何新增规则必须在精确率不下降的前提下再提升召回。

7 结果：AgentDrift 全量重测

7.1 总体指标（结构端与 LLM 端）

表 7-1 AgentDrift test（正 1,056 / 负 825）

口径	P	R	F1	TP	FP	TN	FN
结构 P1–P5（基线）	0.9301	0.7812	0.8492	825	62	763	231
结构 P1–P8（+ 补丁）	**0.9374**	**0.8797**	**0.9077**	929	62	763	127
LLM 旧 rubric（P1–P5）	0.8373	0.8920	0.8638	942	183	642	114
LLM 新 rubric（P1–P7）	0.8733	0.8750	0.8742	924	134	691	132
合并（LLM ∨ 结构 P8）	**0.8794**	**0.9252**	**0.9017**	977	134	691	79

P8 边际（相对结构 P1–P5）：ΔF1 = **+0.0585**、ΔP = **+0.0073**、ΔR = **+0.0985**、ΔFP = **+0**。

7.2 逐子型召回

表 7-2 逐子型命中数（结构 P1–P5 / 结构 P1–P8 / LLM / 合并）

子型	n	结构 P1–P5	结构 P1–P8	LLM	合并
attacked_full	466	446	446	443	443
attacked_partial	231	121	**225**	174	227
attacked_delayed	134	56	56	86	86
failed_attack	225	202	202	221	221
hard_negative (误报)	225	39	39	76	76
benign (误报)	600	23	23	58	58

attacked_partial 召回 **52.4% → 97.4%**（P8 单独贡献），其余子型不变 → 证明 P8 只对 partial 生效且不外溢。

7.3 模板分布

表 7-3 模板分布（AgentDrift 1,881）

来源	P1	P2	P3	P4	P5	P8	signal_only
结构判据	398	288	170	31	861	104	29
LLM 判读	375	321	140	222	818	–	5

LLM 端 P6/P7 触发 0 条（P6 门控在记忆介导，AgentDrift 非记忆轨迹 → 零触发）。结构判据与既有 `a2+P4+s3p` 配置一致性核对：**不一致 0 条。**

7.4 MemEvoBench 结果

0.678
结构 F1（P1–P8）

92.7%
ingestion 召回（127/137）

93.1%
external_send 召回（27/29）

85.9%
记忆节点 E2 连通率

表 7-4　MemEvoBench Workflow（320 条记忆）

口径	P	R	F1
结构 P1–P5（基线）	0.689	0.187	0.294
结构 P1–P8	0.535	**0.928**	**0.678**
LLM 判读（新 rubric）	0.536	0.940	0.683

精确率下降的代价主要来自标签噪声：134 个 `correct` 误报中 55 条（41%）本身含外发/敏感动作。QA 段（324 条误导记忆）无工具动作 → 无 `AC[目标]` 节点，结构检测器不可达，需 LLM 判读（结构检出 0，如实报告）。

8 归因：漏报来自哪、P8 为何命中

8.1 32 条新增漏报的画像

LLM 判读在新 rubric 下由正转负的 32 条样本：

表 8-1 新增漏报的类别与结构模板

维度	分布
类别	attacked_partial 25、attacked_delayed 4、attacked_full 3
结构模板	25/32 判为 P5（无带信号节点）——结构端未识别出注入载体
步级标签	全部含 `injection_point` + `hijacked`，确为真实攻击（非标签噪声）
主攻子型	attacked_partial（注入藏于观察文本，无独立带信号节点）

8.2 候选信号筛选

表 8-2 候选信号对比（负样本 825 条）

信号	含义	负样本误报	覆盖（/32）
SYS	推理引用系统级指令	**0**	23
SYS_WIDE	放宽为 `the system <任意词>`	0	26
COMPLY	伪造合规指令	**32**	–
CRED	凭证 / 危险命令词	**10**	–
3-gram 新颖度	参数 n-gram 新颖度	–	已证伪

> **不采纳 COMPLY / CRED**：F1 仅再 +0.01，但精确率降 1.6–2.3pt，违反"精确率优先"。COMPLY 在良性委托句（"as per <医生/客户> 的请求"）上误报 32 条；CRED 在 coding 良性场景（"enter your credentials"）上误报 10 条。3-gram 新颖度在 hard_negative 上均值最高，无区分度。

8.3 P8 的机制解释

P8 救回的漏检（53 条）**全部为 attacked_partial**，原因是机制性的，而非统计巧合：

1. **注入溶解。**partial 注入藏在观察文本中，没有独立的带信号节点 → 结构端无带信号节点、LLM 也读不到独立注入点 → 双端漏检（结构判 P5、LLM 判 P5）。
2. **伪装为系统指令。**注入措辞伪装成系统级指令，Agent 在推理中把它内化为"系统建议 / 系统要求"。
3. **留下语言痕迹。**这种内化只在 partial 场景发生，体现为 `sys` 信号；其他攻击子型无此语言特征，全部负样本零触发。

因此 P8 是攻击机制的**语言签名**，其触发集合与 attacked_partial 高度重合。

9 探针验证：LLM 判读端对齐

问题：新增 P8 后，结构端能识别 partial 注入，但 LLM 判读端的 rubric 未显式编码该信号。需要验证把 P8 判据写入 rubric 后 LLM 是否会一致翻转。

9.1 路线 C 探针

挑出"结构 P8 命中 ∧ LLM 判 attack=0"的子集，用**含 P8 判据的新 rubric** 重跑（真实调用 53 次）：

53	52/53	1
真实调用	翻转为 attack=1（98.1%）	未翻转（判定尺度差异）

唯一未翻转样本 `web_attacked_partial_0194`，属判定尺度差异而非 rubric 缺失。结论：**rubric 对齐有效。**

9.2 为何最终选路线 B 而非全量重跑

路线 C 证明对齐有效，但全量重跑需大量 API 调用。注意到 `final = LLM_pred ∨ structural_P8` 与"带 P8 判据的 LLM 重跑"几乎等价，且**零 API 调用**，故采纳路线 B（见第 10 章）。

10 结构化合并：LLM ∨ 结构 P8

```
final_pred = LLM_pred ∨ structural_P8
其中 structural_P8 = (轨迹命中 P8 图模式) = 存在 Th 节点满足 sys
```

表 10-1 路线 B 合并效果（零 API 调用）

口径	P	R	F1	TP	FP	TN	FN
LLM（P1–P7）	0.8733	0.8750	0.8742	924	134	691	132
P8（仅 SYS）	1.0000	0.0985	0.1793	104	0	825	952
合并（LLM ∨ P8）	**0.8794**	**0.9252**	**0.9017**	977	134	691	79

边际：**ΔF1 = +0.0275**、**ΔR = +0.0502**、**ΔP = +0.0061**、**ΔFP = +0**。P8 救回 **53 条**漏检，负样本新增误报 **0 条。**

表 10-2 合并前后逐子型召回

子型	n	LLM	合并
attacked_full	466	443	443
attacked_partial	231	174	**227**
attacked_delayed	134	86	86
failed_attack	225	221	221
hard_negative (误报)	225	76	76
benign (误报)	600	58	58

attacked_partial 召回 **75.3% → 98.3%**；其余子型与全部负样本误报不变。

10.1 结构端 vs LLM 端一致率

- AgentDrift：LLM 与结构决策一致 **1,596 / 1,881 = 84.8%。**
- 新旧 rubric 翻转 113 条：旧误报→正确 **58** | 新增误报 **9** | 旧漏报→检出 **14** | 新增漏报 **32。**
- MemEvoBench：LLM 与结构判据高度收敛（仅 9/320 条决策不同）。

11 决策记录与再现性

表 11-1 决策记录

决策	内容	依据
D1 模板记法	弃用早期抽象角色记号，改用「节点类型 × 节点属性 × 边」的原生枚举	表达力（一攻击多模式）+ 与实现对齐
D2 相邻粒度	"相邻"取步骤级（step 差 == 1）	与 E1 步骨架一致，且保持 0 差异
D3 新增 P8	落地"伪装系统指令型"，仅在 P1–P7 未命中时启用	ΔF1 +0.0585、ΔP +0.0073、ΔFP +0
D4 否决 COMPLY/CRED	不纳入	精确率降 1.6–2.3pt，违反精确率优先
D5 对齐路线	采纳路线 B（后置 OR），放弃全量重跑	零 API 调用，与路线 C 探针几乎等价
D6 属性下沉	判定含义下沉为节点属性	2,201 条轨迹 0 差异校验通过
D7 三问输出契约	边收敛为「时间先后 / 信息传导」两类（可并存）；P1–P8 从预枚举模板降级为 Q3 演化路径的压缩标签；新增薄派生层 `_v3_analysis.py`，检测计算零改动	2,201 条上 Q1≡hit、Q2.primary≡min(signal_steps)、Q3.shape↔标签三项全等价，零 API

11.1 再现性

- **零 API 复现。**结构判据、模板分布、合并结果均为纯离线计算，可完全复现。
- **缓存机制。**LLM 判读缓存键 = `sha256(MODEL + SYSTEM_PROMPT + prompt)`；rubric 变更自动使旧缓存失效。
- **口径复用。**规则常量与工具映射复用 `_struct_v2`，避免与既有消融实验产生口径漂移。

12 经验与教训

有效

- **属性下沉 + 图模式**让模板可读、可复现，且不损失性能（0 差异）。
- **"机制签名"优于"统计特征"**：SYS 在 825 条负样本上零误报，是攻击机制的必然痕迹，而非阈值调参的产物。
- **零调用后置逻辑（OR 合并）**在判定差异场景下，可替代昂贵的全量重跑。
- **精确率优先**的规则准入，保证了新信号不引入代价。

无效 / 已否决

- **基于 3-gram 新颖度的检测**无效：hard_negative 的均值最高，无区分度。
- **COMPLY（伪造合规指令）**：在良性委托句上误报 32 条，不适合作为通用规则。
- **CRED（凭证/危险命令词）**：在 coding 良性样本上误报 10 条。

边界与局限

- **P8 只覆盖 partial。**其 `sys` 痕迹是 attacked_partial 注入被内化的机制签名，其他攻击类型不具备该语言特征——这是它的收益来源，也是它的适用边界。
- **attacked_delayed 召回仍低**（结构 41.8%）：延迟触发的注入不产生即时的 `AC[目标]` 节点，现有模板未覆盖。
- **MemEvoBench 精确率**受标签噪声拖累（41% 误报本身含外发/敏感动作）。
- **QA 段结构不可达**：无工具动作 → 无 `AC[目标]` 节点，需依赖 LLM 判读。

13 附录：表示、三问输出与校验的代码落地

13.1 落地位置

节点/边与节点侧判定的实现位于 `v3-defs/_v3_defs.py`；三问输出由**薄派生层** `v3-defs/_v3_analysis.py` 承担（只读 `classify` 结果做步号算术，不改任何检测计算）：

表 13-1 实现结构

函数	职责
`build_nodes(rec, cfg)`	产出 4 类节点，并把判定所需含义下沉为节点属性
`build_edges(nodes)`	E1 相邻骨架 + E2 类型化实体交集（可并存）
`find_signal_nodes(nodes, …)`	求值谓词 A（带信号节点；读节点类型 / 文本属性 / 下游节点），返回 {step: 节点 nid}
`find_target_actions(nodes)`	求值谓词 B（危险动作节点；读 AC 节点的 `unauth_ext/harm/branch3`）
`assign_template(nodes, …)`	P1–P5 的节点/边图模式匹配（步骤级相邻）
`detect_p6/p7/p8(nodes)`	三条低召回补丁的图模式匹配
`classify(rec, cfg)`	统一接口：建图 → 节点侧判定 → 标签 → 返回结果字典
三问输出层（_v3_analysis.py，纯派生，零新判定）	
`analyze(rec)`	由 `classify` 结果派生 `q1`（outcome/severity）、`q2`（primary/secondary 源点）、`q3`（shape/path 演化）
`verify(records)`	三条等价校验：Q1≡hit、Q2.primary≡min(signal_steps)、Q3.shape↔P1–P8

13.2 重构校验（关键证据）

改造采用**忠实改写**策略：新版图模式与基线（`_v3_defs_legacy.py`）应在同一批轨迹上完全一致。校验脚本 `_v3_refactor_check.py` 逐条比较 `hit / template / reason / signal_steps / target_steps / target_detail / 建图统计 / 节点骨架 / 边`：

```
AgentDrift   n = 1,881
MemEvoBench  n =    320
合计 2,201 条，逐条字段差异 0 处
结果：CLEAN —— 图骨架 / 判定 / 模板 / 统计完全一致（纯离线，零 API 调用）
```

该结论同时保证：本报告第 7–10 章的全部指标在改造前后**数值不变。**

三问输出层的等价性由 `_v3_analysis.py verify` 单独核对（同 2,201 条，纯离线）：

```
等价 1  (outcome ≠ none) == hit ..................... AgentDrift 1,881/1,881
MemEvoBench 320/320
等价 2  Q2.primary.step == min(signal_steps) ....... AgentDrift   926/926  MemEvoBench
120/120
等价 3  Q3.shape ⇔ P1–P8 一一对应 .................. AgentDrift 1,881/1,881  MemEvoBench
320/320
结果：三条等价全部成立，检测计算未改动、零 API。
```

13.3 权威数据文件

表 13-2　数据与结果文件

文件	内容
`_v3_summary.txt`	全数据集重测汇总（权威数字来源）
`_v3_p8_decision.txt`	P8 归因决策与逐子型召回
`_v3_p8_or.txt`	路线 B 合并结果
`_v3_refactor_check.py`	表示重构的 0 差异校验脚本
`_v3_analysis.py`	三问输出层（Q1/Q2/Q3 派生）与三条等价校验，零 API
`_v3_eval_agentdrift.py` / `_v3_eval_memevobench.py`	两个数据集的离线评估